Frostlake V1.6 Hardware Architecture & Assembler Manual

24-Bit RISC CPU Microarchitecture Technical Specification

1. Architectural Tier Breakdown

Major Tier (Core Microarchitecture Overhauls)

- Sparse Opcode Layout: Reorganized instruction opcodes into sparse numeric blocks (Block 0 at 0x00, Block 1 at 0x01, Block 2 at 0x10, Block 3 at 0x30, Block 4 at 0x50) to simplify decoding and reserve headroom for pipelining.
- Control Unit Line Decoder: Upgraded from a 6-to-64 decoder to a full 8-to-256 Line Decoder to manage 256 hardware control lines.
- FLOCK Hardware Subsystem: Integrated a hardware flip-flop state register (Opcode 0x57). When enabled, status flags are locked and only CMP/CMPR instructions can update flag states.
- Dynamic Flag Updates by Default: Transformed flag mechanics so all ALU operations update status flags by default (unless FLOCK is active).
- Standalone Assembler (.exe): Compiled the Python toolchain into a standalone .exe executable for direct compilation without local runtime dependencies.

Medium Tier (ALU & Datapath Extensions)

- Hardware Shift Execution (SAR/SAL): Added 1-bit arithmetic right (SAR, 0x18) and arithmetic left (SAL, 0x19) shift instructions, routed directly through the main 8-bit ALU bus and output latch in-place on target register RAQ.
- Extended Carry/Borrow Arithmetic: Added ADC, ADCR, SBB, and SBBR.
- Active Borrow Convention: Configured SBB and SBBR to evaluate $RAQ = RAQ - Operand - C$ using $C = 1$ as an active borrow condition.

Relevant Tier (Control & Register Mapping)

- Control Flow Mapping: Placed FLOCK immediately after RET at Opcode 0x57 in Block 4.
- Control Tunnels: Maintained ROP (Register Operations), IOP (Immediate Operations), and MOP (Memory/Stack Operations).
- In-Place Shift Execution: Configured SAR and SAL to execute strictly on RAQ, bypassing secondary register lookup (RBQ).

Informational Tier (System Parameters & Roadmap)

- Core Parameters: 24-bit instruction word (8:8:8 split), native 8-bit data execution paths, 8-bit address space (256 bytes RAM/ROM), 8 General Purpose Registers (REG A–REG H), and Stack Pointer (SP initialized to 0xFF).
- Pipeline Roadmap: Prepares the microarchitecture for v1.7 (pipeline prep) and v1.8–v2.0 (3-stage Fetch/Decode/Execute pipeline).
- Rebranded the codename from the old Frostlake to the new RISCrunner to reflect the RISC-Like approach.

2. Core Architecture Overview & System Parameters

The Frostlake V1.6 is an upgraded 24-bit RISC microarchitecture engineered for precise deterministic execution, simplified instruction decoding, and expanded control capability.

Parameter	Specification	Microarchitectural Description
Instruction Word Width	24 Bits	Fixed 3-byte layout: Byte 2 = Opcode, Byte 1 = Operand 1 / Immediate / Address, Byte 0 = Operand 0 / Target Register.
Data Bus Width	8 Bits	Native execution paths, ALU processing units, and internal storage buses.
Address Space Capacity	8 Bits	Addresses 256 Bytes of unique RAM/ROM memory locations.
Register File Matrix	8 Registers	8 General Purpose Registers (REG A through REG H, 8-bit native width).
Control Unit Decoder	8-to-256 Line	Upgraded 8-to-256 Line Decoder (expanded from previous 6-to-64 design to support future pipeline expansion).

Parameter	Specification	Microarchitectural Description
Stack Pointer (SP)	8 Bits	Dedicated 8-bit register, automatically initialized to `0xFF` upon hardware reset.

3. Status Flags & FLOCK Control Logic

Status Flags Matrix

The processing unit tracks four core ALU hardware status flags:

- **Zero Flag (Z):** Set when the ALU execution result equals `0x00`.
- **Negative Flag (N):** Set when the most significant bit (MSB, Bit 7) of the ALU result is `1`.
- **Overflow Flag (V):** Set when a two's-complement arithmetic overflow occurs.
- **Carry Flag (C):** Set when an arithmetic operation generates a carry out or a borrow condition.

Dynamic Flag Updates

Unlike architecture version 1.5 where status flags were exclusively updated via explicit comparison instructions, **all ALU operations in v1.6 update flags by default** during every execution cycle.

FLOCK (Flag Lock) Mechanism

To prevent standard arithmetic and logical operations from overwriting critical state flags during multi-step branching or conditional evaluation sequences, V1.6 introduces the `FLOCK` hardware subsystem:

- **Hardware Implementation:** `FLOCK` operates as a dedicated hardware flip-flop state register (Default State: DISABLED / `0`).
- **State Toggling:** Executing the `FLOCK` instruction inverts the flip-flop state (`0` $\rightarrow$ `1`, `1` $\rightarrow$ `0`).
- **Flag Locking Logic:** When `FLOCK` is ENABLED (`1`), all status flags are hardware-locked. During this active lock period, **only `CMP` and `CMPR` instructions are permitted to update flag states**.

Carry & Borrow Convention

Multi-byte and extended arithmetic operations utilize the hardware Carry flag across `ADC`, `ADCR`, `SBB`, and `SBBR` instructions:

- **Addition with Carry:** $RAQ = RAQ + Operand + C$
- **Subtraction with Borrow:** For `SBB` and `SBBR`, $C = 1$ indicates that a **Borrow** occurred during computation ($RAQ = RAQ - Operand - C$).

4. ALU Datapath & Hardware Shift Extensions

Datapath Integration

The shift operations `SAR` (Shift Arithmetic Right) and `SAL` (Shift Arithmetic Left) are routed directly through the primary 8-bit ALU main bus and latched directly into the ALU output latch, eliminating peripheral shift-register routing bottlenecks.

Shift Execution Mechanics

- **Single-Bit Operation:** Shift instructions execute as single-bit hardware shifts (1 bit per instruction cycle).
- **Target Execution:** Shifts act strictly on the designated target register `RAQ` in-place.
- **Arithmetic Sign Preservation:** `SAR` propagates the sign bit (Bit 7) to maintain two's-complement integrity during right-shift execution.

5. Complete V1.6 Sparse Opcode Dictionary

The V1.6 instruction set utilizes a sparse block allocation scheme designed to reserve operational headroom for future microarchitectural pipelines and integrated hardware peripherals.

Block 0: System Control (Opcode 0x00)

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
`0x00`	0	`NOP`	`NOP`	No Operation; advances program counter by 1 cycle.

Block 1: Arithmetic Operations (Opcodes 0x01 - 0x08)

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
`0x01`	1	`ADD`	`ADD RAQ, #imm`	Add Immediate to Target Register
`0x02`	2	`ADDR`	`ADDR RAQ, RBQ`	Add Register `RBQ` to Target Register `RAQ`
`0x03`	3	`ADC`	`ADC RAQ, #imm`	Add Immediate to Target Register with Carry
`0x04`	4	`ADCR`	`ADCR RAQ, RBQ`	Add Register `RBQ` to Target Register `RAQ` with Carry
`0x05`	5	`SUB`	`SUB RAQ, #imm`	Subtract Immediate from Target Register
`0x06`	6	`SUBR`	`SUBR RAQ, RBQ`	Subtract Register `RBQ` from Target Register `RAQ`
`0x07`	7	`SBB`	`SBB RAQ, #imm`	Subtract Immediate from Target Register with Borrow
`0x08`	8	`SBBR`	`SBBR RAQ, RBQ`	Subtract Register `RBQ` from Target Register `RAQ` with Borrow

(Opcodes `0x09` through `0x0F` are reserved for future arithmetic expansion)

Block 2: Logical & Shift Operations (Opcodes 0x10 - 0x19)

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
`0x10`	16	`AND`	`AND RAQ, #imm`	Bitwise AND Immediate with Target Register
`0x11`	17	`ANDR`	`ANDR RAQ, RBQ`	Bitwise AND Register `RBQ` with Target Register `RAQ`
`0x12`	18	`OR`	`OR RAQ, #imm`	Bitwise OR Immediate with Target Register

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
`0x13`	19	`ORR`	`ORR RAQ, RBQ`	Bitwise OR Register `RBQ` with Target Register `RAQ`
`0x14`	20	`NOT`	`NOT RAQ, #imm`	Bitwise NOT Immediate to Target Register
`0x15`	21	`NOTR`	`NOTR RAQ, RBQ`	Bitwise NOT Register `RBQ` to Target Register `RAQ`
`0x16`	22	`CMP`	`CMP RAQ, #imm`	Compare Immediate with Target Register (Updates Flags Only)
`0x17`	23	`CMPR`	`CMPR RAQ, RBQ`	Compare Register `RBQ` with Target Register `RAQ` (Updates Flags Only)
`0x18`	24	`SAR`	`SAR RAQ`	Shift Arithmetic Right (Target Register `RAQ` 1-bit right via ALU)
`0x19`	25	`SAL`	`SAL RAQ`	Shift Arithmetic Left (Target Register `RAQ` 1-bit left via ALU)

(Opcodes `0x1A` through `0x2F` are reserved for future logical and bitwise expansion)

Block 3: Memory & Register Transfer Operations (Opcodes 0x30 - 0x35)

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
`0x30`	48	`LDI`	`LDI RAQ, #imm`	Load 8-bit Immediate into Target Register
`0x31`	49	`LDA`	`LDA RAQ, addr`	Load Target Register from RAM Address
`0x32`	50	`MOV`	`MOV RAQ, RBQ`	Copy contents of Register `RBQ` into Target Register `RAQ`

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
`0x33`	51	`STA`	`STA RAQ, addr`	Store Target Register contents into RAM Address
`0x34`	52	`PUSH`	`PUSH RAQ`	Push Target Register onto Stack (decrements SP)
`0x35`	53	`POP`	`POP RAQ`	Pop top of Stack into Target Register (increments SP)

(Opcodes `0x36` through `0x4F` are reserved for future memory expansion)

Block 4: Control Flow & State Operations (Opcodes 0x50 - 0x57)

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
0x50	80	JMP	JMP addr	Unconditional Branch to Target RAM Address
0x51	81	JE	JE addr	Branch to Target Address if Equal (Z == 1)
0x52	82	JNE	JNE addr	Branch to Target Address if Not Equal (Z == 0)
0x53	83	JGE	JGE addr	Branch to Target Address if Greater or Equal (N == V)
0x54	84	JLE	JLE addr	Branch to Target Address if Less or Equal (Z == 1 or N != V)
0x55	85	CALL	CALL addr	Subroutine Call (Pushes Return Address, jumps to Target)
0x56	86	RET	RET	Subroutine Return (Pops Return Address into Program Counter)

Opcode (Hex)	Opcode (Dec)	Mnemonic	Format	Description
0x57	87	FLOCK	FLOCK	Toggle Flag Lock State Flip-Flop

(Opcodes `0x57` and above are reserved for hardware control expansion)

6. Assembly Program Example

None

```
; Frostlake V1.6 Assembly Program
init:
    LDI REG A, #0xFE    ; Low byte
    LDI REG B, #0x02    ; Addend
    ADDR REG A, REG B   ; 0xFE + 0x02 = 0x00
    FLOCK               ; Status einfrieren
    ADC REG A, #0x00    ; Add with Carry
    SAR REG A           ; Shift Right
    JE init             ; Sprung wenn Z=1
```

7. Standalone Assembler (.exe) User Guide & Tutorial

Quick-Start Execution Steps

1. Prepare Source File: Place your assembly source code file (e.g., main.fsp or program.asm) in the same directory as frostlake_assembler.exe.
2. Launch Assembler: Double-click frostlake_assembler.exe to run the standalone executable.
3. Select/Input Source File: If prompted via GUI/dialog, click Browse (or press [Enter] after typing the filename in CLI mode) to select your .fsp/.asm source file.
4. Compile / Assemble: Click the Assemble / Build button (or press [Enter] to confirm the assembly operation).
5. Load Vector in Logisim:
    a. Open Logisim Evolution v2.0.
    b. Right-click the RAM / ROM memory block in your schematic.
    c. Click Load Image... from the context menu.
    d. Select the newly generated output.hex file to load your 24-bit compiled machine code directly into CPU memory.